

Informe Técnico Proyecto 01: gRPC + docker
Sistema de Casino Online

Christian Gajardo, Alex Saez, Yoandri Villarroel

19 de Octubre, 2026

Repositorio: `github.com/ChGSufro/sis-distribuidos-casino`

Índice

1. Descripcion del Sistema **3**

2. Diagrama de Arquitectura **4**

3. Decisiones Tecnicas y Trade-offs **5**

3.1. Justificacion de Limites de Servicio 5

3.2. Ausencia de Acoplamiento Negativo 5

3.3. Persistencia (Database-per-Service) 5

3.4. Manejo de Dinero y Transacciones en Dos Fases 6

3.5. Comunicacion Interna gRPC frente a REST 6

3.6. Resiliencia ante Fallos Externos 7

4. Descripcion de Servicios y Metodos gRPC **8**

4.1. API Gateway 8

4.2. Auth and Users Service 9

4.3. Wallet Service 10

4.4. Blackjack Game Service 12

4.5. Poker Game Service 13

4.6. History and Audit Service 17

4.7. Leaderboard and Stats Service 18

5. Casos de Uso y Flujos de Comunicacion **20**

5.1. CU-01: Registrar Cuenta 20

5.2. CU-02: Procesar Depósito de Saldo 20

5.3. CU-03: Procesar Partida de Blackjack 21

5.4. CU-04: Obtener Historial y Estadísticas 21

1. Descripcion del Sistema

El Sistema de Casino Online es una plataforma distribuida basada en microservicios desarrollada en lenguaje Go. Su proposito es permitir a los usuarios autenticarse, administrar un balance de creditos virtuales a traves de una billetera digital y participar en partidas de juegos de cartas, especificamente Blackjack y Texas Hold'em Poker, contando con un sistema de auditoria inmutable de eventos y generacion de clasificaciones competitivas en tiempo real.

El sistema esta compuesto por los siguientes actores y componentes:

- **Clientes e Interacción:** El jugador interactúa con el sistema a través de clientes web consumiendo una interfaz HTTP/REST expuesta exclusivamente por el API Gateway.
- **API Gateway:** Opera como el punto de entrada único del clúster, aislando la red interna. Traduce las peticiones HTTP/REST entrantes hacia llamadas remotas gRPC y gestiona de forma centralizada la validación criptográfica de sesiones y tokens JWT.
- **Servicios de Negocio (comunicación interna vía gRPC sobre HTTP/2):**
  - **Auth and Users:** Gestiona el registro de usuarios, almacenamiento de contraseñas mediante hash y control de sesiones activas.
  - **Wallet:** Administra el libro contable de saldos virtuales y ejecuta operaciones transaccionales mediante un protocolo de bloqueo y liquidación de fondos.
  - **Blackjack:** Ejecuta la lógica de juego individual contra el crupier de la casa, soportando apuestas, división de pares, doblaje y rendición.
  - **Poker:** Administra mesas multijugador, control de asientos, asignación de ciegas, etapas de apuestas comunitarias y reparto de botes principales y secundarios.
  - **History and Audit:** Almacena de forma inmutable cada evento relevante de juego y emite un flujo continuo de eventos mediante *gRPC streaming*.
  - **Leaderboard and Stats:** Consume el flujo de eventos de auditoría para mantener agregadas las métricas de rendimiento de cada jugador y publicar tablas de clasificación por periodos de tiempo.

La persistencia de datos sigue estrictamente el patron Database-per-Service, asegurando que ningun servicio acceda directamente a las tablas o colecciones de otro servicio.

2. Diagrama de Arquitectura

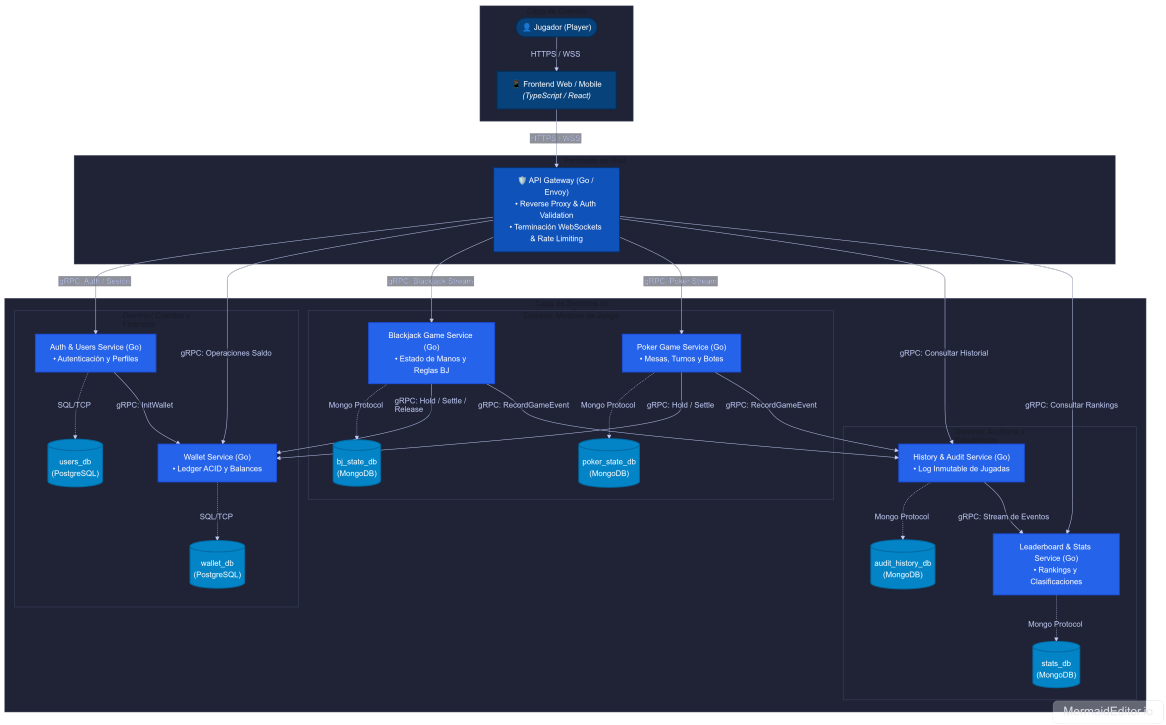


Figura 1: Diagrama de Arquitectura del Sistema

El sistema se despliega en una red interna aislada configurada en Docker Compose denominada casino-net. El unico contenedor que expone puertos al entorno exterior es el API Gateway mediante el puerto HTTP 8080.

Los microservicios de negocio (users, wallet, blackjack, poker, history y stats) junto a sus respectivas bases de datos independientes (dos instancias relacionales en PostgreSQL y cuatro instancias documentales en MongoDB) residen en el perimetro cerrado de la red interna, resolviendo sus conexiones a traves del servicio DNS interno de Docker sin exponer puertos publicos.

3. Decisiones Tecnicas y Trade-offs

3.1. Justificacion de Limites de Servicio

Cada microservicio fue definido en torno a un limite de dominio de negocio, evitando la creacion de un monolito y separando responsabilidades operativas:

Auth and Users se delimita a la identidad, gestion de credenciales y ciclo de vida de sesiones. No gestiona saldo ni reglas de juego. Se descarto unificarlo con el servicio Wallet para no mezclar requerimientos de seguridad y autenticacion con logica transaccional contable.

Wallet se limita al registro contable de creditos virtuales y a garantizar consistencia de saldo. No conoce reglas de juego ni perfiles de usuario. Se descarto que cada juego administre sus propios creditos de forma descentralizada, ya que ello aumentaria el riesgo de dobles gastos e inconsistencias financieras entre partidas concurrentes.

Blackjack opera como motor de partidas individuales sincronas entre un jugador y la casa. Se descarto unificar Blackjack y Poker en un unico servicio de juegos debido a que manejan modelos de concurrencia y estado incompatibles: Blackjack es una maquina de estados determinista de un solo usuario, mientras que Poker requiere coordinacion en tiempo real entre multiples participantes en una misma mesa.

Poker gestiona mesas multijugador persistentes, turnos coordinados, temporizadores de accion y pozos comunitarios. Mantiene su propio ciclo de vida independiente del resto de juegos.

History and Audit centraliza el almacenamiento de auditoria inmutable exclusivamente para operaciones de adicion. Se separo de los motores de juego para evitar que las operaciones de consulta y archivado historico compitan por recursos con la ejecucion de partidas en vivo.

Leaderboard and Stats procesa datos analiticos y genera clasificaciones de forma asincrona a partir de eventos emitidos por el servicio History. Se descarto calcular las estadisticas consultando directamente la base de datos de History, ya que la ejecucion de consultas agregadas intensivas sobre el log transaccional degradaria el rendimiento de escritura de las auditorias.

3.2. Ausencia de Acoplamiento Negativo

Se maneja el **Common Coupling** al prohibir que multiples servicios compartan un mismo motor de base de datos o esquema relacional. Cada servicio tiene su propio contenedor de base de datos y volumen de almacenamiento independiente. Las referencias entre servicios se realizan unicamente a traves de identificadores universales (UUID) transmitidos por llamadas de red.

Se maneja el **Content Coupling** debido a que ningun microservicio puede modificar ni inspeccionar directamente estructuras de datos internas o memoria de otro servicio. Toda comunicacion pasa por contratos explicitos de interfaz definidos en Protocol Buffers.

Se maneja el **Pass-Through Coupling** asegurando que las llamadas directas ocurran entre los servicios que realmente necesitan la operacion, sin que existan servicios intermediarios pasivos que transporten parametros o estructuras que no les competen.

3.3. Persistencia (Database-per-Service)

PostgreSQL en users_db y wallet_db:

Se selecciono un motor relacional tradicional para Auth y Wallet debido a la necesidad de garantias de consistencia ACID, llaves foraneas internas para asegurar integridad referencial, indices unicos en correos o tokens, y validaciones estrictas a nivel de motor de datos como evitar saldos negativos.

Lo que se gana: Integridad absoluta de balances financieros y consistencia de perfiles y sesiones.

Lo que se sacrifica: Mayor rigidez en la modificacion de esquemas, exigiendo migraciones DDL con-

troladas.

Alternativa descartada: Usar MongoDB para Wallet. Fue descartado porque un ledger contable requiere garantias estrictas de atomicidad y bloqueos a nivel de fila para impedir saldo negativo bajo alta concurrencia.

MongoDB en bj_state_db, poker_state_db, audit_history_db y stats_db:

Se selecciono un motor documental NoSQL para los juegos y servicios analiticos debido a la naturaleza dinamica y anidada de sus entidades. En Blackjack, una mano almacena arreglos de cartas y un conjunto variable de sub-manos derivadas de divisiones de cartas. En Poker, una mesa contiene arreglos dinamicos de asientos, botes secundarios y acciones historicas de la ronda. En History, los detalles varian de acuerdo a la accion ejecutada sin requerir columnas nulas. En Stats, las metricas temporales se agrupan en documentos de series de tiempo actualizables atomicamente con operadores de incremento.

Lo que se gana: Alta velocidad en operaciones de escritura append-only y flexibilidad total en estructuras de documentos anidados sin sobrecostos de joins relacionales.

Lo que se sacrifica: Ausencia de transacciones multi-documento cross-collection por defecto y responsabilidad en la aplicacion de validar la coherencia estructural del documento.

Alternativa descartada: Guardar el estado de manos y mesas de juego en tablas relacionales PostgreSQL. Fue descartado debido a que requeriria constantes uniones entre tablas de cartas, acciones y manos en cada turno, elevando innecesariamente la latencia de respuesta del juego.

3.4. Manejo de Dinero y Transacciones en Dos Fases

Protocolo transaccional Hold-Settle-Release con idempotencia:

Al iniciar una mano en Blackjack o pagar el buy-in para sentarse en una mesa de Poker, el motor de juego solicita un Hold a Wallet con un monto y una clave de idempotencia unica. Wallet descuenta el saldo disponible y crea una reserva activa.

Al finalizar el juego, el servicio invoca Settle indicando la reserva y el monto ganado final para liquidar la apuesta e ingresar las ganancias si corresponden.

Si la partida es cancelada o abortada antes de concluir, se invoca Release para anular la reserva y reintegrar el monto bloqueado al balance disponible del usuario.

Este mecanismo desacopla la duracion de una partida de las transacciones de base de datos de Wallet, previniendo doble gasto y tolerando reintentos sin duplicar cargos.

3.5. Comunicacion Interna gRPC frente a REST

La comunicacion inter-servicio utiliza gRPC sobre HTTP/2 con Protocol Buffers 3.

Lo que se gana: Contratos fuertemente tipados verificados durante la compilacion, serializacion binaria de bajo consumo computacional y ancho de banda, multiplexacion de multiples llamadas sobre una sola conexion TCP y capacidades de server-streaming para la emision continua de eventos.

Lo que se sacrifica: Dificultad para inspeccionar el trafico directamente con navegadores o herramientas HTTP convencionales sin decodificacion de protobuf. Por esta razon, el API Gateway actua como traductor de HTTP/REST hacia gRPC.

Alternativa descartada: REST con JSON para comunicacion interna. Fue descartado por el costo de serializacion y parseo repetitivo de cadenas de texto y la ausencia de tipado estricto en tiempo de compilacion entre los servicios.

3.6. Resiliencia ante Fallos Externos

En Blackjack, la comunicacion con el servicio History incluye un timeout acotado y manejo de error con modo degradado. Si History no responde, Blackjack emite una advertencia de registro y procede a completar la liquidacion de la mano y saldo del usuario, priorizando no congelar los fondos del jugador frente a una falla en el subsistema de auditoria.

En Poker, las dependencias hacia Wallet e History estan desacopladas mediante interfaces de clientes, permitiendo la ejecucion de la logica interna y pruebas de forma aislada.

En Stats, el consumidor del stream de History implementa reconexion continua con backoff exponencial sin limite de reintentos. Guarda en base de datos un cursor con el ultimo evento procesado, permitiendo que tras una caida de red o reinicio del servicio se soliciten unicamente los eventos posteriores a dicho cursor, garantizando continuidad sin perdida ni duplicacion de metricas.

4. Descripcion de Servicios y Metodos gRPC

A continuacion se describe cada microservicio implementado y los metodos remotos que expone formalmente en sus archivos de definicion Protocol Buffers.

4.1. API Gateway

El API Gateway es el servidor de entrada perimetral escrito en Go. No expone servicios gRPC propios; consume los servicios gRPC del sistema y expone rutas HTTP/REST hacia los clientes externos:

Rutas publicas de autenticacion y consulta:

- `POST /api/auth/register`: realiza el registro invocando Register en Users y coordinando InitWallet en Wallet.
- `POST /api/auth/login`: valida credenciales en Users y entrega tokens JWT.
- `POST /api/auth/refresh`: renueva el access token mediante RefreshToken en Users.
- `GET /api/games/poker/tables`: lista mesas disponibles consultando ListTables en Poker.
- `GET /api/stats/leaderboard`: consulta rankings invocando GetLeaderboard en Stats.

Rutas protegidas mediante validacion de token JWT:

- `POST /api/auth/logout`: revoca la sesion activa mediante Logout en Users.
- `GET /api/auth/profile` y `GET /api/users/{id}/profile`: consultan datos mediante GetProfile en Users.
- `GET /api/wallet/balance`: obtiene el saldo disponible invocando GetBalance en Wallet.
- `POST /api/wallet/deposit`: recarga fondos en la cuenta mediante Deposit en Wallet.
- `POST /api/games/blackjack/hands`: inicia una mano invocando CreateHand en Blackjack.
- `POST /api/games/blackjack/hands/{id}/action`: ejecuta una jugada mediante PlayerAction en Blackjack.
- `GET /api/games/blackjack/hands/{id}`: consulta el estado de la mano mediante GetHandState en Blackjack.
- `POST /api/games/poker/tables`: crea una nueva mesa mediante CreateTable en Poker.
- `POST /api/games/poker/tables/{id}/join`: une al jugador a la mesa mediante JoinTable en Poker.
- `POST /api/games/poker/tables/{id}/leave`: retira al jugador de la mesa mediante LeaveTable en Poker.
- `POST /api/games/poker/tables/{id}/action`: envia una accion de juego mediante PlayerAction en Poker.
- `GET /api/games/poker/tables/{id}`: consulta el estado de la mesa mediante GetTableState en Poker.
- `GET /api/history`: consulta el historial del jugador mediante GetPlayerHistory en History.
- `GET /api/stats/player`: consulta estadisticas globales mediante GetPlayerStats en Stats.
- `GET /api/stats/player/{game}`: consulta estadisticas de un juego especifico mediante GetPlayerGameStats en Stats.

4.2. Auth and Users Service

El servicio administra la creacion y validacion de cuentas de usuario en users_db. Expone los siguientes metodos gRPC:

```
syntax = "proto3";

package users;

message RegisterRequest {
  string username = 1;
  string email = 2;
  string password = 3;
}

message RegisterResponse {
  string user_id = 1;
  string username = 2;
  string email = 3;
  string created_at = 4;
}

message LoginRequest {
  string email = 1;
  string password = 2;
}

message LoginResponse {
  string access_token = 1;
  string refresh_token = 2;
  string expires_at = 3;
  string user_id = 4;
  string username = 5;
  string status = 6;
}

message ValidateTokenRequest {
  string access_token = 1;
}

message ValidateTokenResponse {
  bool valid = 1;
  string user_id = 2;
  string username = 3;
}

message RefreshTokenRequest {
  string refresh_token = 1;
}

message RefreshTokenResponse {
  string access_token = 1;
  string refresh_token = 2;
  string expires_at = 3;
}

message LogoutRequest {
```

```
  string access_token = 1;
}

message LogoutResponse {
  bool success = 1;
}

message GetProfileRequest {
  string user_id = 1;
}

message GetProfileResponse {
  string user_id = 1;
  string username = 2;
  string email = 3;
  string status = 4;
  string created_at = 5;
  string last_login_at = 6;
}

service UsersService {
  rpc Register (RegisterRequest) returns (RegisterResponse);
  rpc Login (LoginRequest) returns (LoginResponse);
  rpc ValidateToken (ValidateTokenRequest) returns (ValidateTokenResponse);
  rpc RefreshToken (RefreshTokenRequest) returns (RefreshTokenResponse);
  rpc Logout (LogoutRequest) returns (LogoutResponse);
  rpc GetProfile (GetProfileRequest) returns (GetProfileResponse);
}
```

El metodo Register toma el nombre de usuario, correo y contrasena en texto plano. Aplica hashing mediante bcrypt y persiste el nuevo usuario en base de datos devolviendo su identificador UUID y fecha de creacion.

El metodo Login verifica las credenciales recibidas contra el hash almacenado, genera un access token JWT y un refresh token criptografico, registrando la sesion activa.

El metodo ValidateToken es utilizado por el API Gateway para comprobar si un access token posee una firma valida y no ha sido revocado en la base de datos de sesiones.

El metodo RefreshToken valida un refresh token vigente y genera un nuevo par de tokens con rotacion de credenciales.

El metodo Logout revoca la sesion vinculada al access token marcandola en la base de datos.

El metodo GetProfile retorna la informacion publica de la cuenta de usuario identificada por su UUID.

4.3. Wallet Service

El servicio administra el balance de creditos virtuales y las transacciones en wallet_db. Expone los siguientes metodos gRPC:

```
syntax = "proto3";

package wallet;

message InitWalletRequest {
  string user_id = 1;
}

message InitWalletResponse {
```

```
  string wallet_id = 1;
  int64 balance_cents = 2;
}

message GetBalanceRequest {
  string user_id = 1;
}

message GetBalanceResponse {
  string user_id = 1;
  int64 balance_cents = 2;
  string updated_at = 3;
}

message DepositRequest {
  string user_id = 1;
  int64 amount_cents = 2;
}

message DepositResponse {
  string transaction_id = 1;
  int64 new_balance_cents = 2;
}

message HoldRequest {
  string user_id = 1;
  int64 amount_cents = 2;
  string game_type = 3;
  string game_ref_id = 4;
  string idempotency_key = 5;
}

message HoldResponse {
  string hold_id = 1;
  int64 amount_cents = 2;
  string status = 3;
}

message SettleRequest {
  string hold_id = 1;
  int64 win_amount_cents = 2;
}

message SettleResponse {
  string hold_id = 1;
  string status = 2;
  string transaction_id = 3;
  int64 new_balance_cents = 4;
}

message ReleaseRequest {
  string hold_id = 1;
}

message ReleaseResponse {
  string hold_id = 1;
```

```
  string status = 2;
  string transaction_id = 3;
  int64 new_balance_cents = 4;
}

service WalletService {
  rpc InitWallet (InitWalletRequest) returns (InitWalletResponse);
  rpc GetBalance (GetBalanceRequest) returns (GetBalanceResponse);
  rpc Deposit (DepositRequest) returns (DepositResponse);
  rpc Hold (HoldRequest) returns (HoldResponse);
  rpc Settle (SettleRequest) returns (SettleResponse);
  rpc Release (ReleaseRequest) returns (ReleaseResponse);
}
```

El metodo InitWallet crea la billetera contable en estado inicial con saldo cero para el identificador de usuario provisto.

El metodo GetBalance obtiene el saldo disponible en centavos y la estampa de tiempo de su ultima actualizacion.

El metodo Deposit incrementa atomicamente el saldo de la billetera en la cantidad indicada y registra la transaccion contable correspondiente.

El metodo Hold verifica que exista saldo suficiente, lo descuenta de la cuenta del usuario e inserta una reserva activa identificada por una clave de idempotencia unica.

El metodo Settle liquida una reserva previa, marcandola como concluida y acreditando en la cuenta del usuario el monto de ganancias indicado, que puede ser cero si perdio la apuesta.

El metodo Release anula una reserva activa devolviendo de manera integra el monto bloqueado al balance disponible del usuario.

4.4. Blackjack Game Service

El servicio ejecuta las partidas de Blackjack y gestiona el estado de las manos en bj_state_db. Expone los siguientes metodos gRPC:

```
syntax = "proto3";

package blackjack;

message Card {
  string suit = 1;
  string rank = 2;
  int32 value = 3;
  bool hidden = 4;
}

message SubHand {
  int32 index = 1;
  repeated Card cards = 2;
  int64 bet_amount_cents = 3;
  string status = 4;
  string result = 5;
}

message CreateHandRequest {
  string user_id = 1;
  int64 bet_amount_cents = 2;
}
```

```
message CreateHandResponse {
  string hand_id = 1;
  repeated SubHand sub_hands = 2;
  Card dealer_visible_card = 3;
  string status = 4;
}

message PlayerActionRequest {
  string hand_id = 1;
  string user_id = 2;
  int32 sub_hand_index = 3;
  string action = 4;
}

message PlayerActionResponse {
  string hand_id = 1;
  repeated SubHand sub_hands = 2;
  repeated Card dealer_cards = 3;
  string status = 4;
  int64 total_win_amount_cents = 5;
}

message GetHandStateRequest {
  string hand_id = 1;
  string user_id = 2;
}

message GetHandStateResponse {
  string hand_id = 1;
  repeated SubHand sub_hands = 2;
  repeated Card dealer_cards = 3;
  string status = 4;
  int64 total_win_amount_cents = 5;
}

service BlackjackService {
  rpc CreateHand (CreateHandRequest) returns (CreateHandResponse);
  rpc PlayerAction (PlayerActionRequest) returns (PlayerActionResponse);
  rpc GetHandState (GetHandStateRequest) returns (GetHandStateResponse);
}
```

El metodo CreateHand genera una nueva partida para el usuario, ejecuta un Hold por el monto apostado en el servicio Wallet, baraja y reparte dos cartas al jugador y dos al crupier (una visible y una oculta), almacenando el documento de la mano en MongoDB.

El metodo PlayerAction procesa jugadas como pedir carta (hit), plantarse (stand), doblar apuesta (double), dividir pares (split) o rendirse (surrender). Al concluir las acciones del jugador, ejecuta el turno del crupier hasta que este alcance 17 puntos o mas, determina los resultados, ejecuta Settle en Wallet por cada sub-mano e invoca el registro del evento en History.

El metodo GetHandState permite consultar en cualquier momento la situacion actual de una mano en curso o terminada.

4.5. Poker Game Service

El servicio administra mesas y rondas de Texas Hold'em en poker_state_db. Expone los siguientes metodos gRPC:

```
syntax = "proto3";

package poker;

message Card {
  string suit = 1;
  string rank = 2;
}

message Table {
  string table_id = 1;
  string name = 2;
  int64 small_blind_cents = 3;
  int64 big_blind_cents = 4;
  int32 max_players = 5;
  int32 current_players = 6;
  string status = 7;
}

message Seat {
  int32 seat_position = 1;
  string user_id = 2;
  int64 stack_cents = 3;
  string status = 4;
}

message PlayerState {
  string user_id = 1;
  int32 seat_position = 2;
  int64 stack_cents = 3;
  int64 total_bet_in_round_cents = 4;
  string status = 5;
}

message Action {
  string user_id = 1;
  string action = 2;
  int64 amount_cents = 3;
  string timestamp = 4;
}

message SidePot {
  int64 amount_cents = 1;
  repeated string eligible_players = 2;
}

message Winner {
  string user_id = 1;
  string pot_type = 2;
  int64 amount_cents = 3;
}

message RoundState {
  string round_id = 1;
  string stage = 2;
  repeated Card community_cards = 3;
```

```
  repeated Card your_hole_cards = 4;
  int64 pot_size_cents = 5;
  int64 current_bet_cents = 6;
  int32 current_turn = 7;
  string turn_expires_at = 8;
  int32 dealer_position = 9;
  repeated Action actions = 10;
  repeated SidePot side_pots = 11;
}

message TableUpdate {
  string round_id = 1;
  string stage = 2;
  int32 current_turn = 3;
  int64 current_bet_cents = 4;
  int64 pot_size_cents = 5;
  repeated Card community_cards = 6;
  repeated PlayerState players = 7;
  string turn_expires_at = 8;
  repeated Winner winners = 9;
  int32 dealer_position = 10;
  repeated Action actions = 11;
  repeated SidePot side_pots = 12;
}

message ListTablesRequest {
  string status_filter = 1;
}

message ListTablesResponse {
  repeated Table tables = 1;
}

message CreateTableRequest {
  string user_id = 1;
  string name = 2;
  int64 small_blind_cents = 3;
  int64 big_blind_cents = 4;
  int32 max_players = 5;
}

message CreateTableResponse {
  Table table = 1;
}

message JoinTableRequest {
  string table_id = 1;
  string user_id = 2;
  int64 buy_in_amount_cents = 3;
  int32 preferred_seat = 4;
}

message JoinTableResponse {
  string table_id = 1;
  int32 seat_position = 2;
  int64 stack_cents = 3;
```

```
  string hold_id = 4;
}

message LeaveTableRequest {
  string table_id = 1;
  string user_id = 2;
}

message LeaveTableResponse {
  string table_id = 1;
  int64 final_stack_cents = 2;
  int64 settled_amount_cents = 3;
}

message PlayerActionRequest {
  string table_id = 1;
  string round_id = 2;
  string user_id = 3;
  string action = 4;
  int64 amount_cents = 5;
}

message PlayerActionResponse {
  TableUpdate update = 1;
}

message GetTableStateRequest {
  string table_id = 1;
  string user_id = 2;
}

message GetTableStateResponse {
  Table table = 1;
  repeated Seat seats = 2;
  RoundState current_round = 3;
}

service PokerService {
  rpc ListTables (ListTablesRequest) returns (ListTablesResponse);
  rpc CreateTable (CreateTableRequest) returns (CreateTableResponse);
  rpc JoinTable (JoinTableRequest) returns (JoinTableResponse);
  rpc LeaveTable (LeaveTableRequest) returns (LeaveTableResponse);
  rpc PlayerAction (PlayerActionRequest) returns (PlayerActionResponse);
  rpc GetTableState (GetTableStateRequest) returns (GetTableStateResponse);
}
```

El metodo ListTables retorna las mesas existentes con soporte de filtrado segun su estado operativo.

El metodo CreateTable registra una nueva mesa persistente configurando nombre, valor de ciegas y maximo de jugadores.

El metodo JoinTable asigna un asiento disponible en la mesa para un jugador e inicia la reserva de sus fichas iniciales (buy-in) mediante Hold en Wallet.

El metodo LeaveTable remueve al jugador de su asiento y liquida su acumulado de fichas final mediante Settle en Wallet, reingresando los creditos a su balance.

El metodo PlayerAction procesa apuestas en la ronda activa (fold, check, call, raise o all-in), administra botes secundarios y transiciona las etapas de la partida (preflop, flop, turn, river y showdown), evaluando ganadores al concluir la mano.

El metodo GetTableState consulta la situacion global de la mesa, asegurando que las cartas privadas de otros jugadores no sean reveladas antes del showdown.

4.6. History and Audit Service

El servicio almacena el registro inmutable de auditoria en audit_history_db y difunde eventos mediante streaming. Expone los siguientes metodos gRPC:

```
syntax = "proto3";

package history;

message GameEvent {
  string event_id = 1;
  string user_id = 2;
  string game_type = 3;
  string game_ref_id = 4;
  string action = 5;
  string details = 6;
  string timestamp = 7;
  string cursor = 8;
}

message RecordGameEventRequest {
  string event_id = 1;
  string user_id = 2;
  string game_type = 3;
  string game_ref_id = 4;
  string action = 5;
  string details = 6;
  string timestamp = 7;
}

message RecordGameEventResponse {
  bool recorded = 1;
  string event_id = 2;
}

message GetPlayerHistoryRequest {
  string user_id = 1;
  string game_type = 2;
  int32 limit = 3;
  int32 offset = 4;
}

message GetPlayerHistoryResponse {
  repeated GameEvent events = 1;
  int32 total_count = 2;
}

message StreamGameEventsRequest {
  string cursor = 1;
}

service HistoryService {
  rpc RecordGameEvent (RecordGameEventRequest) returns (RecordGameEventResponse);
```

```
  rpc GetPlayerHistory (GetPlayerHistoryRequest) returns (GetPlayerHistoryResponse);
  rpc StreamGameEvents (StreamGameEventsRequest) returns (stream GameEvent);
}
```

El metodo RecordGameEvent recibe eventos provenientes de los motores de juego y los persiste de manera inmutable en la coleccion game_events, distribuyendolos al broker interno en memoria.

El metodo GetPlayerHistory retorna una lista cronologica y paginada de las jugadas y apuestas efectuadas por un jugador determinado.

El metodo StreamGameEvents abre un canal de transmision server-streaming unidireccional. Si se provee un cursor de identificador documental, transmite en primer termino los eventos historicos ocurridos con posterioridad a dicho cursor y continua luego emitiendo eventos en vivo.

4.7. Leaderboard and Stats Service

El servicio mantiene agregadas las estadisticas de los jugadores y las tablas de posiciones en stats_db. Expone los siguientes metodos gRPC:

```
syntax = "proto3";

package stats;

message LeaderboardEntry {
  int32 rank = 1;
  string user_id = 2;
  string username = 3;
  int64 score_cents = 4;
  int32 games_played = 5;
}

message GetPlayerStatsRequest {
  string user_id = 1;
}

message GetPlayerStatsResponse {
  string user_id = 1;
  int64 total_wagered_cents = 2;
  int64 total_won_cents = 3;
  int64 net_profit_cents = 4;
  int32 games_played = 5;
  double win_rate = 6;
  int64 largest_win_cents = 7;
  int32 current_streak = 8;
  int32 best_streak = 9;
  string last_updated = 10;
}

message GetPlayerGameStatsRequest {
  string user_id = 1;
  string game_type = 2;
}

message GetPlayerGameStatsResponse {
  string user_id = 1;
  string game_type = 2;
  int32 games_played = 3;
  int64 total_wagered_cents = 4;
```

```
  int64 total_won_cents = 5;
  int64 largest_win_cents = 6;
  string last_updated = 7;
}

message GetLeaderboardRequest {
  string period = 1;
  int32 limit = 2;
}

message GetLeaderboardResponse {
  string period = 1;
  string period_start = 2;
  repeated LeaderboardEntry entries = 3;
  string computed_at = 4;
}

service StatsService {
  rpc GetPlayerStats (GetPlayerStatsRequest) returns (GetPlayerStatsResponse);
  rpc GetPlayerGameStats (GetPlayerGameStatsRequest) returns (GetPlayerGameStatsResponse);
  rpc GetLeaderboard (GetLeaderboardRequest) returns (GetLeaderboardResponse);
}
```

El metodo GetPlayerStats retorna los indicadores acumulados de un jugador consolidando el total de sus partidas (apuestas totales, saldo neto, victorias, rachas y mayor premio).

El metodo GetPlayerGameStats obtiene las mismas metricas analiticas pero segmentadas especificamente por tipo de juego.

El metodo GetLeaderboard retorna la lista de jugadores con mejor desempeno ordenados segun su ganancia neta en centavos en un intervalo de tiempo establecido (diario, semanal, mensual o historico).

5. Casos de Uso y Flujos de Comunicacion

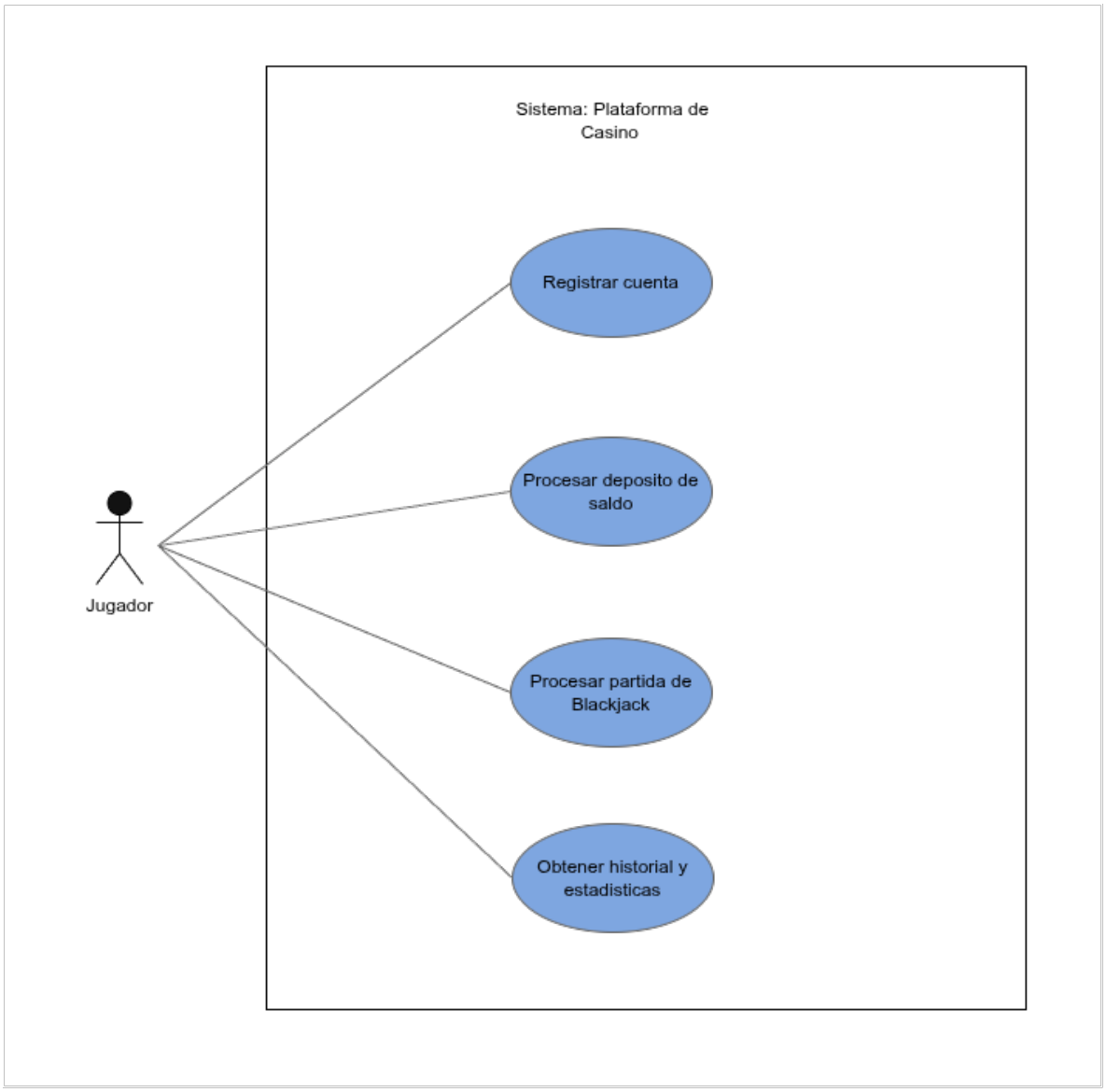


Figura 2: Casos de Uso del Sistema

5.1. CU-01: Registrar Cuenta

El usuario solicita la creación de una cuenta desde el cliente web enviando sus credenciales. El API Gateway recibe la petición y coordina de forma secuencial la invocación gRPC a **Auth and Users** para almacenar el usuario con su contraseña cifrada en PostgreSQL (`users_db`), y luego a **Wallet** mediante `InitWallet` para inicializar su libro contable con saldo cero en `wallet_db`.

5.2. CU-02: Procesar Depósito de Saldo

El jugador autenticado envía una solicitud para recargar créditos virtuales en su cuenta. El API Gateway valida el token JWT con **Auth and Users** y redirige la petición gRPC al servicio **Wallet**, el cual ejecuta una operación en PostgreSQL (`wallet_db`) para incrementar el saldo disponible y registrar la transacción contable correspondiente.

5.3. CU-03: Procesar Partida de Blackjack

El jugador inicia y desarrolla una mano de cartas contra la casa. El servicio **Blackjack** solicita una reserva temporal de saldo mediante el método `Hold` en **Wallet**, gestiona las cartas y turnos persistiendo el estado en MongoDB (`bj_state_db`) y, al determinar el resultado, invoca `Settle` en **Wallet** para liquidar las ganancias y emite el evento a **History and Audit** para su registro inmutable.

5.4. CU-04: Obtener Historial y Estadísticas

El jugador consulta sus jugadas pasadas y su rendimiento acumulado. El API Gateway obtiene el historial cronológico llamando a **History and Audit** sobre `audit_history_db`, y consulta las métricas consolidadas (victorias, apuestas y rachas) en **Leaderboard and Stats**, servicio que procesa y mantiene agregados estos datos en MongoDB (`stats_db`) consumiendo el flujo de eventos en tiempo real.